05

WEEK 5

4 units · ~32 pages

Tractable Structure and Symmetry

U1 Tree-Structured CSPs and Directional Arc Consistency

U2 Cutset Conditioning, Tree Decomposition and Treewidth

U3 Complexity: What Makes a CSP Hard

U4 Symmetry and Dominance

Chapter Summary

Check Yourself Answers

Learning Objectives

By the end of this chapter, you will be able to:

- **(H1)** recognize a tree-structured CSP from its constraint graph, apply directional arc consistency (DAC) with respect to a tree-consistent variable ordering, and explain why the resulting forward assignment is guaranteed backtrack-free;
- **(H2)** explain cutset conditioning and tree decomposition as two ways of handling a constraint graph that is *not* a tree, compute the induced width of a small constraint graph for a given elimination ordering, and connect that number to the graph's treewidth;
- **(H3)** explain, in terms of constraint-graph structure, what makes a CSP tractable versus NP-hard in the worst case, and relate bounded-treewidth tractability to the general hardness result – including why Week 4's search engineering (restarts, nogood learning, local search) improves practical performance without changing this worst-case picture;

- **(H4)** identify variable and value symmetry in a CSP instance, and apply symmetry-breaking constraints – including a lexicographic ordering constraint – in an OR-Tools CP-SAT model to eliminate redundant, symmetric regions of the search space without removing any essentially different solution;
- **(H5)** build and run CP-SAT models of a classic combinatorial-design problem (social golfer or BIBD) both with and without symmetry-breaking constraints, and interpret the solver's own search statistics as empirical evidence for the structural ideas taught across this chapter.

Weeks 1–4 built models (Week 1), propagated them (Week 2), searched them systematically (Week 3), and then searched them with local moves, restarts and learning (Week 4). Every one of those four weeks treated the search itself as the place where hardness is fought. This chapter asks a different question, one that never depends on how you search at all: does the problem's own shape make it easy or hard, independent of the algorithm you point at it? The answer has two halves that mirror each other closely. The first half, Units 1–2, is *tractable structure*: some constraint graphs – trees, and more generally graphs of bounded treewidth – admit solving algorithms that never need to backtrack, or need only a controlled, bounded amount of it. The second half, Unit 4, is *symmetry*: even a genuinely hard instance often contains many symmetric copies of the same underlying solution, and a search that does not know this wastes effort rediscovering them one by one. Unit 3 sits between the two halves and ties them together, asking in general what makes a CSP hard at all, and why Week 4's engineering answers – restarts, nogood learning, local search – do not contradict this week's theoretical one; they answer a different question about a different kind of instance.

Two problems drive this chapter's code, and both appear for the first and only time in this course: the *social golfer problem* and *balanced incomplete block design (BIBD)*. Both are classic, heavily symmetric combinatorial-design CSPs – social golfer has golfer-relabeling, group-relabeling and week-relabeling symmetries; BIBD has point-relabeling (row) and block-relabeling (column) symmetries – chosen specifically to make Unit 4's symmetry-breaking constraints show a real, measurable effect. N-queens and map coloring, this course's running problems since Week 1,

reappear only as the source of Unit 1's contrasting example: a genuine tree-shaped map against Week 1's cyclic Australia map.

This chapter stays deliberately narrow in a few places. It introduces no propagation algorithm beyond directional arc consistency – full AC-3/GAC mechanics were Week 2's subject and are only recalled here, not re-taught. It introduces no new general-purpose search heuristic (MRV, dom/wdeg, restarts, large neighbourhood search); this week's "search" content is specifically the *backtrack-free* search that DAC on a tree enables, and the *reduced* search that symmetry breaking enables, both framed as consequences of structure rather than as new entries in Week 3–4's heuristic toolbox. It uses no MiniZinc and no Gurobi – this chapter stays CP-SAT only, exactly as Weeks 1–4 did, with those other tools returning in Weeks 6 and 7 respectively. And it proves no formal NP-completeness reduction in full; Unit 3 states the tractability and hardness results and reuses Week 1's CSP-versus-SAT contrast for intuition, without deriving a formal proof.

5.1 Tree-Structured CSPs and Directional Arc Consistency

Why It Matters 1 · Pre-instruction

In Week 3 you learned to survive a search tree that might, in the worst case, force you to backtrack exponentially many times. This unit shows you a class of problems – recognizable from their constraint graph alone, before you write a single line of search code – where that worst case is *impossible*: if your problem's constraint graph happens to be a tree, one directional sweep and one forward pass is all you will ever need.

Prior Knowledge 1 · Pre-instruction

Week 1's constraint graph (one node per variable, one edge per binary constraint – the same picture behind the Australia map-coloring figure) and Week 3's backtracking search tree are both assumed here; this unit asks you to look at the *first* picture to predict something about the *second*, before you ever build it.

Week 1 drew a constraint graph purely as bookkeeping: a node per variable, an edge per binary constraint, useful for seeing which variables interact but not, so far, used to predict anything about how hard the problem would be to search. This unit puts

that same picture to work. It turns out that one simple, checkable property of the graph's *shape* – whether it is a tree – already tells you, before any search runs, that backtracking will never be needed at all.

5.1.1 Tree-structured CSPs

tree-structured CSP: A CSP whose constraint graph is a tree: connected, and with no cycles.

A CSP is **tree-structured CSP** if its constraint graph is a tree: every pair of nodes is connected by exactly one path, and no sequence of edges ever loops back on itself. Contrast this with Week 1's Australia map-coloring instance (Figure 1.1): its constraint graph has a triangle among Western Australia, Northern Territory and South Australia, three mutually adjacent regions, so tracing WA–NT–SA–WA closes a cycle, and the graph is not a tree.

Definition: Tree-structured CSP

A CSP whose constraint graph is a tree: connected, and with no cycles.

Analogy

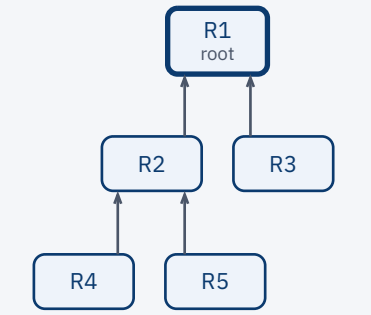
A tree-structured CSP resembles a course prerequisite chain with no cycles: if every course depends on exactly one earlier course, and there is a single root course with no prerequisite at all, you can choose a course order by walking the chain once, root to leaves, and you will never need to undo an earlier choice. The *shape* of the prerequisite structure, not luck, is what guarantees you never get stuck partway through.

This week's own code instance,

```
1 REGIONS = ["R1", "R2", "R3", "R4", "R5"]
2 EDGES = [("R1", "R2"), ("R1", "R3"), ("R2", "R4"),
  ("R2", "R5")]
3
4 def build_domains() -> dict[str, list[int]]:
5     return {region: [0, 1, 2] for region in REGIONS}
```

builds exactly this kind of graph on purpose: five map regions, R_1 through R_5, each with a 3-colour domain, connected by exactly four edges – R_1–R_2, R_1–R_3, R_2–R_4, R_2–R_5 – one fewer edge than there are nodes, and no cycle anywhere among them, so this is genuinely tree-structured, deliberately built to contrast with Week 1's cyclic Australia map. Figure 5.1 draws this exact graph,

rooted at R_1, with the directional processing order this unit's algorithm follows marked on every edge.



A tree-structured constraint graph — contrast with Week 1's Australia map, which has a triangle (WA–NT–SA) and is not a tree.

Figure 5.1: A tree-structured constraint graph for `tree_csp.py`'s 5-region instance, with directional arc consistency's child-to-parent processing direction marked – contrast with Week 1's Australia map (Figure 1.1), whose WA–NT–SA triangle is not a tree.

5.1.2 A tree-consistent ordering

Before any processing can begin, the tree needs a starting point and a direction to walk in. A **tree-consistent ordering** of the variables is an ordering in which every variable, after the first, has exactly one earlier neighbour in the constraint graph: its *parent* in the tree rooted at the ordering's first variable. R_1, R_2, R_3, R_4, R_5 is one tree-consistent ordering for the graph above: R_1 is the root (no earlier neighbour at all); R_2's only earlier neighbour is R_1; R_3's only earlier neighbour is R_1; R_4's only earlier neighbour is R_2; R_5's only earlier neighbour is R_2. Every later variable has exactly one earlier neighbour, which is precisely what the definition requires.

tree-consistent ordering: A variable ordering in which every variable, after the first, has exactly one earlier neighbour in the constraint graph – its parent in the tree, rooted at the ordering's first variable.

Definition: Tree-consistent ordering

A variable ordering in which every variable, after the first, has exactly one earlier neighbour in the constraint graph – its parent in the tree, rooted at the ordering's first variable.

A common point of confusion here is treating "tree-consistent ordering" as if it meant "any ordering of a tree-structured CSP's variables." It does not: an ordering that visited R_4 before its parent R_2, for instance, would give R_4 zero earlier neighbours rather than exactly one, breaking the definition. A tree-consistent ordering always exists for a tree-structured graph – pick any node as root and list the rest by increasing distance from it – but not every listing of the same variables qualifies.

Directional arc consistency (DAC): Arc consistency achieved in one direction only, with respect to a fixed ordering, by processing variables from the last to the second and, for each one, removing from its parent's domain every value with no compatible value left in that variable's own (current) domain.

5.1.3 Directional arc consistency (DAC)

Directional arc consistency (DAC) processes the variables from the *last* one back to the *second* one in a tree-consistent ordering, and for each one revises its parent's domain: remove from the parent's domain every value that has no compatible value left in the child's current domain. Read that direction carefully – DAC only ever tightens a *parent* against an already-processed *child*, never the reverse, which is exactly what the word "directional" names, in contrast with Week 2's AC-3, which revises arcs in both directions until nothing changes anywhere.

Definition: Directional arc consistency (DAC)

Arc consistency achieved in one direction only, with respect to a fixed ordering, by processing variables from the last to the second and, for each one, removing from its parent's domain every value with no compatible value left in that variable's own (current) domain.

Analogy

Directional arc consistency resembles finalizing a multi-leg travel itinerary leg by leg, working backward: processing the very last leg first, then the second-to-last, and so on back to the first leg, tightening each earlier leg's possible departure times based on what the later legs already require. By the time you plan the first leg forward, every remaining choice is already guaranteed to work with everything downstream – you never have to revisit a leg you already fixed.

5.1.4 A worked example: DAC and a backtrack-free forward pass

Apply DAC to the tree-consistent ordering R_1, R_2, R_3, R_4, R_5 above. Processing runs from the last variable back to the second – R_5, R_4, R_3, R_2, in that order – revising each one's *parent* in turn:

```
def apply_dac(domains: dict[str, list[int]]) -> None:
    for child in reversed(ORDER[1:]):
        parent = PARENT[child]
        domains[parent] = [
            value
            for value in domains[parent]
            if any(is_compatible(value, other) for
other in domains[child])
```

```
        ]
        print(f"Revise {parent} against {child}:
  domains[{parent}] = {domains[parent]}")
```

Every region here starts with the same 3-colour domain, so no revision actually removes a value on this particular instance – a perfectly legitimate outcome; DAC narrows domains only when a narrowing is genuinely forced, and a uniform, unconstrained-looking domain like this one is exactly the case where it correctly does nothing. The recorded trace confirms this directly, one revision per non-root region, processed last-to-second exactly as the definition requires:

Expected Output

```
Revise R2 against R5: domains[R2] = [0, 1, 2]
Revise R2 against R4: domains[R2] = [0, 1, 2]
Revise R1 against R3: domains[R1] = [0, 1, 2]
Revise R1 against R2: domains[R1] = [0, 1, 2]
```

With DAC's sweep complete, a *forward* assignment – root to leaves, following the same tree-consistent ordering, picking any remaining domain value at each step – is what **Freuder's tree theorem** guarantees will never need to backtrack:

Definition: Freuder's tree theorem

If a tree-structured CSP's variables are processed in a tree-consistent order after achieving directional arc consistency with respect to that same order, then assigning variables forward, root to leaves, choosing any remaining domain value at each step, never needs to backtrack.

Freuder's tree theorem: If a tree-structured CSP's variables are processed in a tree-consistent order after achieving DAC with respect to that same order, then assigning variables forward, root to leaves, choosing any remaining domain value at each step, never needs to backtrack.

The code confirms this on the real instance, assigning root to leaves and counting every undone assignment along the way, then cross-checking the whole result against an independent, plain CP-SAT solve of the same instance:

```
def forward_assign_and_check() -> dict[str, int]:
    domains = build_domains()
    apply_dac(domains)
    assignment: dict[str, int] = {}
    backtracks = 0
    for region in ORDER:
        parent = PARENT.get(region)
        if parent is None:
```

```
            candidates = domains[region]
        else:
            candidates = [v for v in domains[region]
if is_compatible(v, assignment[parent])]
        if not candidates:
            backtracks += 1
        assignment[region] = candidates[0]
        print(f"Assign {region} =
{COLOUR_NAMES[assignment[region]]}")
    print(f"Backtracks needed: {backtracks}")
    status, cp_sat_assignment =
cross_check_with_cp_sat()
    cp_sat_valid = all(cp_sat_assignment[a] !=
cp_sat_assignment[b] for a, b in EDGES)
    print(f"CP-SAT cross-check status: {status}, valid
coloring: {cp_sat_valid}")
    return assignment
```

Expected Output

```
Assign R1 = Red
Assign R2 = Green
Assign R3 = Green
Assign R4 = Red
Assign R5 = Red
Backtracks needed: 0
CP-SAT cross-check status: OPTIMAL, valid coloring: True
```

Zero backtracks, exactly as Freuder's theorem promises – and notice $R_2 =$ Green and $R_3 =$ Green is perfectly fine here, since R_2 and R_3 are both children of R_1 but are not themselves adjacent in the constraint graph; only edges in the graph constrain a pair of regions to differ, and R_2–R_3 is not one of this tree's four edges. The independent CP-SAT cross-check confirms the hand-verified result is not a coincidence of this particular run.

Caution

Freuder's theorem is not a claim that search becomes unnecessary in general – it is a claim about *this specific combination*: a tree-structured graph, a tree-consistent ordering, and DAC achieved with respect to that same ordering, all three together. Change any one of the three – a non-tree graph, an ordering that is not tree-consistent, or a forward pass attempted without first achieving DAC – and the backtrack-free guarantee no longer applies. Unit 2 asks

exactly what happens once the first of these three conditions fails.

Now You Try 3 · Participation

A 4-node constraint graph has edges A–B, B–C, B–D (a star centred on B) – no other edges. Is this graph tree-structured? Give a tree-consistent ordering of A, B, C, D, and name which single variable DAC would need to revise last (i.e., first in reverse-order processing).

Feedback. Yes, a star is a tree: it is connected, has 3 edges for 4 nodes, and contains no cycle. One tree-consistent ordering is B, A, C, D – B first, as root; each later variable has exactly one earlier neighbour, B, as its parent. Processing in reverse order for DAC visits D first, then C, then A, each time revising B's domain against that one child; B itself, the root, is never revised. After the sweep, an assignment B → A → C → D, choosing any remaining value at each step, is guaranteed backtrack-free.

Check Yourself 5.1

A 4-node constraint graph has edges P–Q, Q–R, R–S (a simple path). Is this graph tree-structured? Give a tree-consistent ordering and state, without running any code, whether forward assignment after DAC filtering can ever need to backtrack.

Transfer — Where Else? 5 · Follow-through

This is the last unit in the book that treats a constraint graph purely by its shape, with no numbers attached. Unit 2 keeps the same picture but asks what to do when it is *not* a tree; Unit 4's social golfer and BIBD instances will turn out to have constraint graphs far from tree-shaped, which is exactly why they need Unit 4's different remedy (symmetry breaking, not DAC) – two problems can both be "hard by default" for entirely different structural reasons.

5.2 Cutset Conditioning, Tree Decomposition and Treewidth

Why It Matters 1 · Pre-instruction

Real constraint graphs are rarely as clean as Unit 1's star or its 5-region tree – most have at least one cycle. This unit answers the practical question that follows immediately: your graph is not a tree, but is it "close enough" to one that you can still avoid the worst case, and if so, by how much extra work?

Prior Knowledge 1 · Pre-instruction

Unit 1's tree-structured CSP algorithm – DAC plus a backtrack-free forward assignment, guaranteed by Freuder's tree theorem – is assumed here. Cutset conditioning and tree decomposition both work by reducing to, or generalizing, exactly that algorithm; neither is a new solving idea built from scratch.

Unit 1 left one loose thread deliberately: what happens once the constraint graph has a cycle, so no tree-consistent ordering exists at all? This unit answers that question two ways – one that removes just enough of the graph to make the rest tree-structured, and one that reformulates the entire graph into a tree of small, overlapping pieces – and then gives the single number that measures how much extra work either approach costs.

5.2.1 Cutset conditioning

Given a constraint graph that is *almost* a tree, a **cutset (cycle cutset)** is a set of variables whose removal leaves a tree behind. In the 5-node example this unit works through below, removing just one well-chosen variable is enough to break every cycle in the graph, which is exactly what makes a cutset worth naming on its own: it is the smallest "price of admission" back to Unit 1's tree-only algorithm. That admission price has its own technique built around it: **cutset conditioning** solves the original problem by fixing a value for each cutset variable in turn, then solving the remaining tree-structured CSP backtrack-free – using exactly Unit 1's algorithm – for each fixed combination of cutset values.

cutset conditioning: Solving a non-tree CSP by fixing values for a cutset's variables and solving the remaining tree-structured problem backtrack-free for each fixed combination.

Definition: Cutset conditioning

Solving a non-tree CSP by fixing values for a cutset's variables and solving the remaining tree-structured problem backtrack-free for each fixed combination.

Analogy

Cutset conditioning resembles a lab with one shared microscope: if several otherwise-independent lab groups' schedules only ever conflict over that one shared microscope, provisionally fixing which group uses the microscope at which time turns one tangled shared-resource schedule into several small, independent, easy-to-solve group schedules – conditioning on just that one shared resource is enough to untangle everything else.

The cutset's size directly controls the cost: a cutset of size c means at most $|D|^c$ tree-structured sub-problems to solve, each one handled backtrack-free once its cutset values are fixed, where $|D|$ is the (largest) domain size.

Complexity (big-O)

Cutset conditioning's total cost is $O(|D|^c \cdot n \cdot |D|^2)$ for a cutset of size c over n variables, where $|D|^c$ counts the fixed-value combinations tried over the cutset and the $n \cdot |D|^2$ factor is Unit 1's own backtrack-free algorithm applied once per combination. For a *fixed*, small c this stays close to linear in n; the entire practical value of cutset conditioning depends on finding a small cutset, since $|D|^c$ grows exponentially in c itself.

5.2.2 Tree decomposition

A more general reformulation groups the constraint graph's variables into overlapping *bags*, arranged in a tree, so that Unit 1's tree-CSP algorithm generalizes to *any* constraint graph, not only ones with a small cutset. A **tree decomposition** arranges the variables into bags connected in a tree such that (i) every constraint's variables all appear together in at least one bag, and (ii) any variable shared by two bags also appears in every bag on the tree-path between them – the *running-intersection property*. Solving bag by bag, passing consistent partial solutions between neighbouring bags along the tree, plays exactly the role Unit 1's

tree decomposition: A tree of overlapping variable "bags," covering every constraint and satisfying the running-intersection property, used to generalize tree-CSP solving to arbitrary constraint graphs.

forward pass played over single variables, but now over whole bags.

Definition: Tree decomposition

A tree of overlapping variable "bags," covering every constraint and satisfying the running-intersection property (any variable shared by two bags also appears in every bag on the tree-path between them), used to generalize tree-CSP solving to arbitrary constraint graphs.

Analogy

Tree decomposition resembles clustering an exam timetable's overlapping constraints into small groups: instead of solving one timetable with hundreds of interacting exams at once, group the exams into small overlapping clusters, each small enough to solve by hand, connected in a tree by their shared exams, and stitch neighbouring clusters together along those shared exams to solve the whole timetable. The size of the largest cluster you needed is, essentially, the number this unit builds toward next.

5.2.3 Induced width and treewidth

That number comes from simulating variable *elimination* in a chosen order: process variables one at a time, and each time a variable is eliminated, connect its still-active (not-yet-eliminated) neighbours into a clique with new "fill-in" edges, since those neighbours are now indirectly linked through the variable just removed. The **induced width** of a given elimination ordering is the largest number of still-active neighbours any single variable has at the moment it is eliminated.

induced width: Given an elimination ordering, the largest number of still-active neighbours any variable acquires as earlier variables are eliminated one at a time.

Definition: Induced width

Given an elimination ordering, the largest number of still-active neighbours any variable acquires as earlier variables are eliminated one at a time.

Induced width is a property of one particular elimination ordering, not of the graph alone – a badly chosen ordering can inflate it well above what the graph's true shape requires, and a well-chosen ordering recovers the smallest value the graph allows. That smallest achievable value is worth naming on its own, since

it no longer depends on which ordering happened to be tried: **treewidth** is the *minimum* induced width over every possible elimination ordering of the graph – equivalently, one less than the largest bag size in the best tree decomposition. A tree, as Unit 1's own graph confirms below, always has treewidth 1; the triangle in Week 1's Australia map already forces treewidth at least 2.

Definition: Treewidth

The minimum induced width over all possible elimination orderings of a constraint graph.

5.2.4 Worked example: a 5-node cutset instance

```
def induced_width(adjacency: Adjacency, order: list[str]) -> int:
    graph = {node: set(neighbours) for node, neighbours in adjacency.items()}
    width = 0
    for node in order:
        neighbours = graph[node]
        width = max(width, len(neighbours))
        for neighbour in neighbours:
            graph[neighbour].update(neighbours - {neighbour})
        for neighbour in neighbours:
            graph[neighbour].discard(node)
        del graph[node]
    return width
```

Applied first to Unit 1's own tree, eliminating leaves before their parent before the root, no elimination step ever finds more than one still-active neighbour:

```
def treewidth_tree_example() -> int:
    width = induced_width(TREE_GRAPH, TREE_ORDER)
    print(f"Elimination order: {TREE_ORDER}")
    print(f"Induced width: {width}")
    return width
```

Expected Output

```
Elimination order: ['R3', 'R4', 'R5', 'R2', 'R1']
Induced width: 1
```

An induced width of 1 for a tree is not a coincidence of this particular ordering: a tree's every non-root node has exactly one parent, so eliminating a leaf can never need a fill-in edge, and this

is why trees always achieve treewidth 1 – the smallest possible value for any graph with at least one edge.

Now take a genuinely non-tree graph: a 5-node constraint graph with edges A–B, B–C, C–D, D–A (a 4-cycle) plus one more edge A–E. Removing A alone breaks the cycle A–B–C–D–A into the path B–C–D (the edge to E simply disappears with A), leaving a tree – so $\{A\}$ is a cutset of size 1 for this graph. Eliminating A first connects its three remaining neighbours, B, D and E, into a clique via three fill-in edges (B–D, B–E, D–E), since all three were only linked to each other through A; A's degree at the moment of its own elimination is therefore 3:

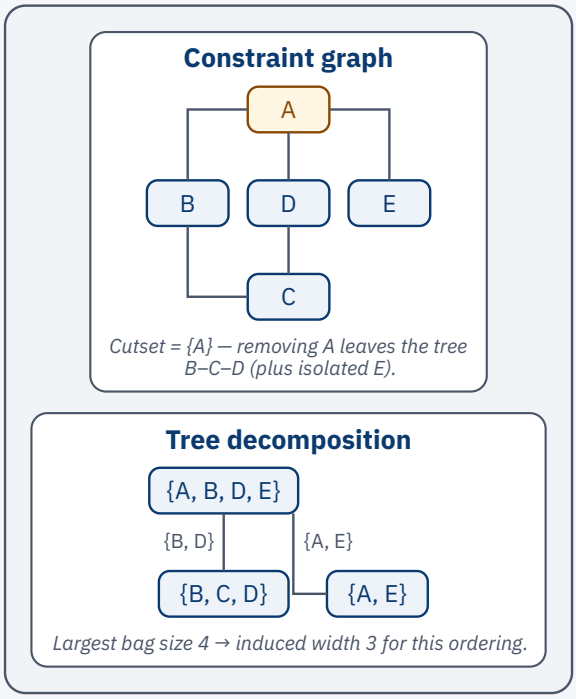
```
def treewidth_cutset_example() -> int:
    width = induced_width(CUTSET_GRAPH, CUTSET_ORDER)
    print(f"Cutset variable eliminated first:
{CUTSET_ORDER[0]}")
    print(f"Elimination order: {CUTSET_ORDER}")
    print(f"Induced width: {width}")
    return width
```

Expected Output

```
Cutset variable eliminated first: A
Elimination order: ['A', 'B', 'D', 'E', 'C']
Induced width: 3
```

Eliminating the rest of the graph in the order B, D, E, C afterward never again reaches a still-active neighbour count above 3, so the induced width for this ordering is 3 – one worked example of a single cutset variable's removal showing up directly in the induced-width computation: cutset conditioning's cutset and tree decomposition's induced width are two ways of measuring the same underlying gap between this graph and a genuine tree.

Figure 5.2 draws this exact instance two ways at once: as a constraint graph with the cutset variable A highlighted, and as the corresponding tree decomposition, three overlapping bags in a small tree – a central bag $\{A, B, D, E\}$ (size 4, covering the A–B, D–A and A–E constraints together) with two leaf bags attached, $\{B, C, D\}$ (covering B–C and C–D, sharing $\{B, D\}$ with the central bag) and $\{A, E\}$ (covering A–E again, sharing $\{A, E\}$ with the central bag). The largest bag has size 4, and one less than the largest bag size is exactly the induced width just computed by elimination: $4 - 1 = 3$, the same number two different ways.



Same graph, two algorithms: condition on the cutset, or decompose into bags.

Figure 5.2: Unit 2's worked 5-node cutset example, shown two ways: as a constraint graph with cutset variable A highlighted (left), and as the corresponding tree decomposition, three overlapping bags in a small tree (right) – cutset conditioning and tree decomposition are two views of the same underlying structural fact.

Common Pitfall

A frequent mistake here is assuming the cutset's size and the induced width must always be numerically identical. They are closely linked – a small cutset tends to produce a small induced width, and eliminating a cutset variable first is a natural strategy – but they are not the same quantity by definition: the cutset is a set of *variables* whose removal leaves a tree, while the induced width is a property of an *elimination ordering*, counting fill-in-inflated neighbour counts. In the worked example above the cutset has size 1 (just $\{A\}$) while the induced width is 3, precisely because eliminating A creates three fill-in edges among its former neighbours – the two numbers answer related but different questions about the same graph.

Now You Try 3 · Participation

A 5-node constraint graph has edges A–B, B–C, C–D, D–A (a 4-cycle) plus one more edge A–E. Name one single variable whose removal leaves a tree, and give the resulting elimination ordering's induced width using that variable last – i.e., the cutset variable eliminated first.

Feedback. Removing A breaks the 4-cycle A–B–C–D–A into the path B–C–D (the now-isolated edge to E vanishes along with A), a tree. Eliminating A first connects its remaining neighbours B, D and E into a clique – three fill-in edges: B–D, B–E, D–E – giving A a degree of 3 at elimination time; eliminating the rest in the order B, D, E, C thereafter never exceeds that number, so the induced width for this ordering is 3.

Check Yourself 5.2

A constraint graph is a single 6-node cycle (A–B–C–D–E–F–A, no other edges). Name a cutset of size 1 and give the induced width achieved by eliminating that cutset variable first.

Transfer — Where Else? 5 · Follow-through

Treewidth is not just a Week 5 curiosity: it is the same number that determines how much a solver's internal search (Weeks 3–4) is fighting an unavoidably tangled graph versus a nearly-tree-shaped one – Unit 3 makes this connection to worst-case hardness explicit, and Unit 4's social golfer and BIBD instances are deliberately dense, high-treewidth graphs, which is part of why they need a different tool (symmetry breaking) rather than a cheap tree-decomposition solve.

5.3 Complexity: What Makes a CSP Hard

Why It Matters 1 · Pre-instruction

Week 4 showed you empirically that CP-SAT is fast on large N-queens and Golomb-ruler instances thanks to restarts, nogood learning and portfolios – but "usually fast in practice" is not the same claim as "always fast," and this

unit tells you, precisely, when a worst case is unavoidable no matter how good your search engineering is.

Prior Knowledge 1 · Pre-instruction

Week 1's CSP-versus-SAT contrast – SAT as the special case of a CSP with Boolean domains and clause constraints – is this unit's hardness argument, made precise. Week 4's restarts, nogood learning and lazy clause generation are also assumed here; this unit deliberately revisits *why* those techniques help in practice without claiming they change the worst case.

Units 1 and 2 gave you two genuinely good pieces of news: a tree-structured graph never needs backtracking at all, and a graph of small treewidth needs only a controlled, bounded amount of it. Neither result claims this holds for *every* CSP – and this unit states, precisely, where the good news stops and a real worst case begins, and why that worst case is not a contradiction of anything Week 4 already showed you working well in practice.

5.3.1 CSP in general is NP-hard

Recall from Week 1 that the Boolean satisfiability problem (SAT) is the special case of a CSP in which every domain is forced down to $\{0, 1\}$ and every constraint is forced into the shape of a logical clause. Since SAT is already known to be NP-complete, and SAT sits inside the wider CSP family as one restricted special case, the general CSP – with arbitrary finite domains and arbitrary constraint relations – is at least as hard as SAT. This makes CSP in general **NP-hard**: no algorithm is known, or, under standard complexity assumptions, believed to exist, that solves every CSP instance in time polynomial in the number of variables.

NP-hard: A problem at least as hard as every problem in NP; no known algorithm solves every instance in polynomial time.

Definition: NP-hard

A problem at least as hard as every problem in NP; no known algorithm solves every instance in polynomial time.

A common misconception at this point is treating "CSP is NP-hard" as a claim about *every single instance* you will ever meet. It is not: NP-hardness is a claim about the *worst case* over the whole family of CSP instances – some instances (Unit 1's trees, for example) are easy in the strongest possible sense, solvable without a single backtrack. NP-hardness only says that no algorithm

handles *every* instance efficiently; it says nothing at all about how a particular, structured instance in front of you will behave.

5.3.2 Bounded treewidth: a genuine tractable island

Unit 2's tree-decomposition algorithm already hints at where the worst-case picture breaks in your favour. For a *fixed* treewidth k, that same algorithm solves a CSP in time polynomial in the number of variables n – though exponential in k itself – by solving the tree decomposition's bags one at a time, exactly as Unit 2 described. This makes bounded-treewidth CSPs a genuine **tractable problem class**: not merely fast in practice, but provably polynomial-time for any fixed bound on k. Unit 1's tree-structured CSPs, with treewidth exactly 1, are simply the simplest member of this same general result, not a separate phenomenon requiring its own separate proof.

tractable problem class: A class of problems solvable in time polynomial in the input size, e.g. CSPs of fixed, bounded treewidth.

Definition: Tractable problem class

A class of problems solvable in time polynomial in the input size, e.g. CSPs of fixed, bounded treewidth.

Analogy

NP-hardness versus bounded-treewidth tractability resembles reconciling department budgets in a fully cross-referenced spreadsheet versus an org-chart-shaped one: if every department's budget request depends on every other department's (a fully tangled constraint graph), checking all the interactions can blow up combinatorially as more departments are added. If each department only overlaps with its direct neighbour on the org chart (a tree, or a graph close to one), the same reconciliation stays manageable no matter how many departments there are.

Complexity (big-O)

Contrast the two cost pictures directly. Unit 2's tree-decomposition algorithm costs roughly $O\big(n \cdot |D|^{k+1}\big)$ for treewidth k over n variables with domain size $|D|$ – polynomial in n whenever k is held fixed, since $|D|^{k+1}$ is then just a constant multiplier. Plain worst-case CSP solving, with no structural guarantee at all, costs $O(|D|^n)$ in the worst case – exponential directly in the number of variables, with no fixed exponent to hide the growth behind. The entire tractability result rests on k staying fixed

as n grows; if k grows along with n (as it does for a dense, tangled graph), the tree-decomposition cost collapses back toward the same exponential shape as plain worst-case solving.

5.3.3 Other structural levers, without the same guarantee

Treewidth is not the only property of a CSP instance that affects how hard it is in practice. **Constraint tightness** – how large a fraction of value combinations a single constraint forbids – and graph density – how many constraints touch each variable – both shape typical solving effort: a tightly constrained instance prunes faster during search, and a sparse instance has fewer interactions to check at each node. But neither comes with a matching worst-case guarantee the way treewidth does; a very tight or very sparse instance can still, in the worst case, force exponential search, and a genuinely dense instance is not automatically hard if its treewidth happens to stay small regardless. Treewidth is the one structural lever this course connects to a proven worst-case bound; tightness and density affect typical-case behaviour without changing that bound one way or the other.

Constraint tightness: How restrictive an individual constraint is, i.e., how large a fraction of value combinations it forbids.

Definition: Constraint tightness

How restrictive an individual constraint is, i.e., how large a fraction of value combinations it forbids.

5.3.4 Why Week 4's engineering does not contradict this result

Local search, restarts, nogood learning and lazy clause generation (Week 4) make many *practical* instances fast without lowering the *worst-case* complexity of CSP in general. An instance with genuinely unbounded treewidth and no exploitable symmetry can still force exponential search time, no matter how well-engineered the search underneath it is – Week 4's techniques improve how quickly a solver typically reaches a good answer on the instances it actually meets, not the provable worst-case bound over the entire space of possible instances.

Common Pitfall

A tempting but incorrect conclusion is: "CP-SAT solved this hard-looking instance quickly, so CSP must not really be NP-hard after all." This confuses a specific instance's practical behaviour with the family's worst-case guarantee. Week 4's restarts and nogood learning are real, measured

wins on the instances they were tested on – nothing in this unit disputes that – but a single fast solve, or even many fast solves, is not a proof that *every* CSP instance solves quickly; it takes exactly one instance with unbounded treewidth and no exploitable symmetry to keep the worst-case NP-hardness result standing, regardless of how many easier instances a well-engineered solver handles well.

Structure (this unit, and Units 1–2 before it) and search engineering (Week 4) are therefore complementary answers to "why is CP-SAT usually fast," not competing ones: structure explains the provably easy cases – trees, bounded treewidth, and Unit 4's symmetric instances – while search engineering explains good typical-case behaviour on everything else, the much larger set of instances with no such guarantee to lean on.

Now You Try 3 · Participation

Two CSP instances have the same number of variables. Instance X has a constraint graph that is a tree; instance Y's constraint graph is a dense graph with treewidth equal to roughly one third of the number of variables. Which one does the tree-decomposition algorithm (Unit 2) guarantee to solve in time polynomial in the number of variables, and why does the same guarantee not apply to the other?

Feedback. Instance X (treewidth 1, fixed and small) is guaranteed polynomial-time solvable by Unit 2's algorithm, since the exponential cost term depends only on the fixed treewidth, not on the number of variables. Instance Y's treewidth grows *with* the number of variables (roughly a third of it), so the "exponential in treewidth" cost term grows right along with the problem size instead of staying fixed – the polynomial-time guarantee only holds for a *fixed* treewidth, not one that scales with the instance.

Check Yourself 5.3

A CSP instance's constraint graph has treewidth exactly equal to the number of variables minus one (i.e., essentially a complete graph). Does the tree-decomposition algorithm from Unit 2 guarantee a polynomial-time solve for this instance? Why or why not?

Transfer — Where Else? 5 · Follow-through

This unit's structural lens generalizes past this course's own CSP problems: whenever you meet a new combinatorial problem in your own research, sketching its constraint graph and asking "is this close to a tree, or hopelessly tangled" is a five-minute check that tells you, before you write any search code, whether to expect a fast exact solve or to reach for the approximate/local-search tools Week 4 already gave you.

5.4 Symmetry and Dominance

Why It Matters 1 · Pre-instruction

Unit 3 told you some problems are hard no matter what – but "hard" often hides an enormous amount of pure repetition: many of a hard instance's "different" solutions are the exact same schedule with the golfers, groups or weeks simply relabeled. This unit shows you how to tell a solver, once, "stop rediscovering relabeled copies of solutions you have already implicitly found" – and how much search space that one instruction removes.

Prior Knowledge 1 · Pre-instruction

Week 1's viewpoint/auxiliary-variable/implied-constraint modeling triad is assumed here: symmetry-breaking constraints are a fourth modeling tool in that same family. Like an implied constraint, a symmetry-breaking constraint changes no essential solution, but unlike an implied constraint (which follows logically from the model already built), a symmetry-breaking constraint deliberately *removes* redundant solutions on purpose.

Every problem this course has modeled so far – N-queens, map coloring, Sudoku – already had symmetry lurking in it without this book naming it: swap two colour names in a map-coloring solution and you get another equally valid solution, for instance. This unit finally names that pattern directly, on two problems chosen specifically because their symmetry is large enough to measure: social golfer and BIBD.

5.4.1 Symmetry in a CSP

symmetry: A transformation of a solution (relabeling variables, values, or both) that always produces another solution to the same instance.

A **symmetry** is a transformation of a solution – relabeling variables, relabeling domain values, or both – that always produces another solution to the same instance. Two common kinds recur across this course's problems. **Value symmetry** arises from interchangeable domain values: in map coloring, swapping "colour 1" and "colour 2" everywhere turns any solution into another equally valid one, since the colours' names never mattered to any constraint in the first place. **Variable symmetry** arises from interchangeable variables instead: in social golfer, relabeling "week 2" and "week 3" throughout turns any valid schedule into another equally valid one, since nothing in the model distinguishes one week's position from another's.

Definition: Symmetry

A transformation of a solution (relabeling variables, values, or both) that always produces another valid solution to the same instance.

Why symmetry hurts search is a direct consequence of this definition: an unmodified search rediscovers every symmetric copy of the same underlying solution – or the same underlying failure – separately, multiplying the search space by the size of the symmetry group without adding anything genuinely new to find. Social golfer's instance below has week-relabeling, group-relabeling and golfer-relabeling symmetries all at once; BIBD has point-relabeling (row) and block-relabeling (column) symmetries. None of this redundancy is visible in either problem's raw constraints – it only shows up once you ask "how many of these 'different' solutions are actually the same schedule wearing a different label."

Analogy

Symmetry resembles four interchangeable teaching-assistant slots for the same lab session: if the four TA slots are otherwise identical, a search that does not know this tries every one of the 24 ways of relabeling "TA1" through "TA4" for what is, underneath, the exact same real assignment. Agreeing in advance that the TAs are always listed in alphabetical order removes 23 of those 24 redundant relabelings without changing which real assignments remain possible.

5.4.2 Symmetry-breaking constraints

A **symmetry-breaking constraint** is an additional constraint, added to an already-correct model, that keeps at least one representative of every symmetry class of solutions while removing the redundant symmetric copies. The one non-negotiable requirement is in that same sentence: a symmetry-breaking constraint must never remove *every* solution in a symmetry class, only the duplicates – removing every last one would silently turn a feasible instance into an infeasible-looking one, an error far worse than leaving the redundancy in.

symmetry-breaking constraint: An added constraint that removes redundant, symmetric duplicate solutions while keeping at least one representative of every symmetry class.

Definition: Symmetry-breaking constraint

An added constraint that removes redundant, symmetric duplicate solutions while keeping at least one representative of every symmetry class.

The most common device for building such a constraint is the **lexicographic ordering constraint**: it forces a sequence of interchangeable objects – rows of a matrix, groups within a week, weeks within a schedule – into one canonical order by requiring each one to be lexicographically less-than-or-equal to the next. This is exactly the device this unit's code applies to both the social golfer and BIBD instances below.

lexicographic ordering constraint: A symmetry-breaking constraint requiring one sequence of variables to be lexicographically less than or equal to another, canonicalizing an otherwise-interchangeable group.

Definition: Lexicographic ordering constraint

A symmetry-breaking constraint requiring one sequence of variables to be lexicographically less than or equal to another, canonicalizing an otherwise-interchangeable group.

Analogy

Lexicographic ordering resembles keeping a reading list alphabetized: many different orderings of the same set of books describe the same underlying reading list, but insisting the list is always written in alphabetical order fixes exactly one of those orderings as "the" canonical one. A lexicographic-ordering constraint does the same thing to an otherwise-interchangeable group of model variables.

5.4.3 Worked example: social golfer

This unit's first coded instance is a small social golfer schedule: 3 groups of 3 golfers (9 golfers total) over 4 weeks, so that no

two golfers ever share a group more than once – 4 weeks is this instance's maximum, since it uses up all $\binom{9}{2} = 36$ pairs exactly once. Every golfer plays exactly one group each week:

```
def build_variables(model: cp_model.CpModel) ->
dict[tuple[int, int, int], cp_model.IntVar]:
    play = {
        (week, golfer, group):
model.NewBoolVar(f"play_w{week}_g{golfer}_grp{group}")
        for week in range(NUM_WEEKS)
        for golfer in range(NUM_GOLFERS)
        for group in range(NUM_GROUPS)
    }
    for week in range(NUM_WEEKS):
        for golfer in range(NUM_GOLFERS):
            model.AddExactlyOne(play[week, golfer,
group] for group in range(NUM_GROUPS))
    return play
```

Each pair of golfers is then constrained to share a group in at most one week, the model's only real constraint beyond the group-size and one-group-per-week bookkeeping already shown above:

```
def add_pairing_constraints(model: cp_model.CpModel,
play) -> None:
    for golfer_a in range(NUM_GOLFERS):
        for golfer_b in range(golfer_a + 1,
NUM_GOLFERS):
            meets_this_week = []
            for week in range(NUM_WEEKS):
                meet = model.NewBoolVar(f"meet_w{week}
_g{golfer_a}_g{golfer_b}")
                for group in range(NUM_GROUPS):
                    model.AddBoolOr(
                        [play[week, golfer_a,
group].Not(), play[week, golfer_b, group].Not(), meet]
                    )
                meets_this_week.append(meet)
            model.Add(sum(meets_this_week) <= 1)
```

Nothing built so far distinguishes one week from another, or one group from another within a week – which is exactly the source of this instance's variable symmetry. The lexicographic-ordering constraints below fix week 1's grouping to one canonical assignment, pin the golfer with the smallest id to group 0 in every other week, and force weeks after week 1 into non-decreasing lexicographic order against each other:

```python
def add_symmetry_breaking(model: cp_model.CpModel,
play) -> None:
    for golfer in range(NUM_GOLFERS):
        model.Add(play[0, golfer, golfer // GROUP_SIZE]
== 1)
    for week in range(1, NUM_WEEKS):
        model.Add(play[week, 0, 0] == 1)
    for week in range(1, NUM_WEEKS - 1):
        left = [_group_of(play, week, golfer) for
golfer in range(NUM_GOLFERS)]
        right = [_group_of(play, week + 1, golfer) for
golfer in range(NUM_GOLFERS)]
        _add_lex_le(model, left, right)
```

Both variants – with and without these constraints – are solved with the same helper, reading off CP-SAT's own search statistics:

```python
solver = cp_model.CpSolver()
solver.parameters.num_workers = 1
solver.parameters.random_seed = 42
status = solver.Solve(model)
print(f"Status: {solver.StatusName(status)}")
print(f"NumBranches: {solver.NumBranches()}")
print(f"NumConflicts: {solver.NumConflicts()}")
print(f"WallTime: {solver.WallTime():.6f}")
```

Without symmetry breaking, the real captured run:

Expected Output

```
Status: OPTIMAL
NumBranches: 2352
NumConflicts: 147
WallTime: 0.011725
```

With symmetry breaking added, the same instance, same solver settings:

Expected Output

```
Status: OPTIMAL
NumBranches: 1454
NumConflicts: 25
WallTime: 0.006005
```

Read this pair honestly: both variants confirm the schedule is feasible, but the symmetry-broken version explored barely more than half as many branches (1454 against 2352) and less

than a fifth as many conflicts (25 against 147). Nothing about *which* schedules remain reachable changed – every essentially different schedule is still findable – only the redundant relabeled copies of each one were removed before the search ever had to visit them.

5.4.4 Worked example: BIBD

The second instance is a balanced incomplete block design (BIBD), the Fano plane, CSPLib's smallest classic case: a 7×7 Boolean incidence matrix, every point in exactly 3 blocks, every block holding exactly 3 points, and every pair of points sharing exactly 1 block:

```
def build_variables(model: cp_model.CpModel) -> dict[tuple[int, int], cp_model.IntVar]:
    x = {
        (point, block): model.NewBoolVar(f"x_{point}_{block}")
        for point in range(NUM_POINTS)
        for block in range(NUM_BLOCKS)
    }
    for point in range(NUM_POINTS):
        model.Add(sum(x[point, block] for block in range(NUM_BLOCKS)) == BLOCKS_PER_POINT)
    for block in range(NUM_BLOCKS):
        model.Add(sum(x[point, block] for point in range(NUM_POINTS)) == POINTS_PER_BLOCK)
    return x
```

The pairwise-sharing requirement is what gives BIBD its name and its difficulty: every pair of points' shared-block count is linearized as a Boolean product per candidate block, then summed and pinned to exactly 1:

```
def add_pairwise_lambda_constraints(model: cp_model.CpModel, x) -> None:
    for point_a in range(NUM_POINTS):
        for point_b in range(point_a + 1, NUM_POINTS):
            shared_blocks = []
            for block in range(NUM_BLOCKS):
                both = model.NewBoolVar(f"both_{point_a}_{point_b}_{block}")
                model.AddMultiplicationEquality(both, [x[point_a, block], x[point_b, block]])
                shared_blocks.append(both)
            model.Add(sum(shared_blocks) == PAIR_COUNT)
```

Nothing distinguishes one block (column) from another here either – BIBD's own block-relabeling symmetry. A lexicographic-ordering constraint between every pair of adjacent block-columns fixes one canonical column order:

```
1 def add_symmetry_breaking(model: cp_model.CpModel, x) -
  > None:
2     for block in range(NUM_BLOCKS - 1):
3         left = [x[point, block] for point in
  range(NUM_POINTS)]
4         right = [x[point, block + 1] for point in
  range(NUM_POINTS)]
5         _add_lex_le(model, left, right)
```

The same `Status`/`NumBranches`/`NumConflicts`/ `WallTime` report used above prints BIBD's own real captured numbers (`kod_ornekleri/hafta05/modeller/bibd.py`, the `bibd_solve_and_report` tag). Without symmetry breaking:

Expected Output

```
Status: OPTIMAL
NumBranches: 1185
NumConflicts: 0
WallTime: 0.005635
```

With symmetry breaking:

Expected Output

```
Status: OPTIMAL
NumBranches: 599
NumConflicts: 0
WallTime: 0.005082
```

The branch count roughly halves again – 599 against 1185 – on a second, structurally different combinatorial-design problem, real evidence that the lexicographic-ordering recipe is not a one-problem trick.

Figure 5.3 makes the same idea visible on a toy instance small enough to draw completely by hand, before either real solver-statistics comparison above: three fully interchangeable Boolean variables have a naive search tree with 8 leaves, four of which are pure relabelings of the other four; a single lexicographic-ordering constraint prunes exactly those redundant relabelings, leaving the same four essentially different assignments behind.

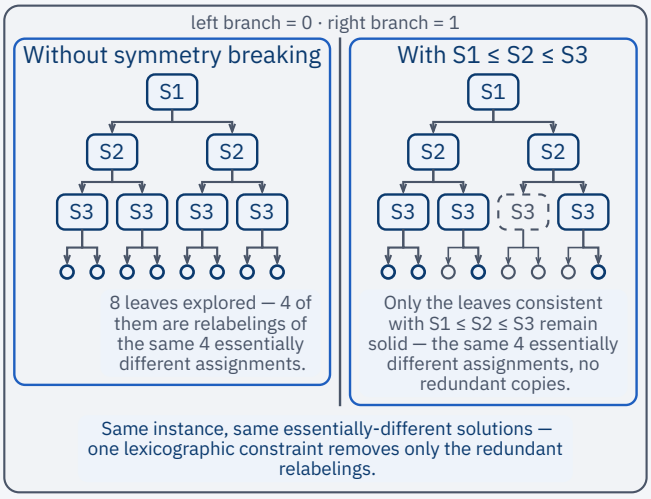


Figure 5.3: A schematic three-variable search tree without symmetry breaking (left, 8 leaves) against the same tree with the lexicographic constraint $S_1 \leq S_2 \leq S_3$ applied (right, redundant relabelings pruned) – illustrative, not a capture of this unit's actual CP-SAT runs above.

Common Pitfall

A dangerous mistake when writing a symmetry-breaking constraint by hand is accidentally over-constraining the model – forcing an order so strict that it removes an entire symmetry class rather than just its duplicates. A lexicographic-ordering constraint like $\text{left} \leq \text{right}$ is safe precisely because it is a *non-strict* inequality: at least one relabeling of any symmetric solution already satisfies it (the smallest one, in lexicographic order), so the class always keeps a representative. A strict version, $\text{left} < \text{right}$, would incorrectly discard any solution whose two halves happened to be lexicographically equal – always double-check that a hand-written symmetry-breaking constraint keeps room for equality exactly where the original model allowed it.

Neither instance's symmetry-breaking constraints above are exhaustive: social golfer's groups within a non-canonical week can still be relabeled against each other, and this is deliberate – this unit's scope is naming and applying the lexicographic-ordering idea, not engineering a competition-grade, exhaustive symmetry-group quotient. A strict generalization of this whole idea, named here and not built, is **dominance detection**: instead of requiring solutions to be literally identical under a relabeling, dominance detection prunes a partial assignment whenever it

dominance detection: A generalization of symmetry breaking that prunes a partial assignment whenever it is provably no better than another already explored, without requiring an exact symmetry.

is provably *no better than* another partial assignment already explored, even without an exact symmetry between them.

Definition: Dominance detection

A generalization of symmetry breaking that prunes a partial assignment whenever it is provably no better than another already explored, without requiring an exact symmetry.

Now You Try 3 · Participation

Three interchangeable review slots (Slot1, Slot2, Slot3) must each be assigned one of three reviewers (X, Y, Z), one reviewer per slot, with no constraint distinguishing the slots from each other. Write one lexicographic-ordering-style constraint on the slot variables that removes the pure slot-relabeling symmetry without eliminating any genuinely different reviewer assignment.

Feedback. Since the slots are fully interchangeable, one standard fix is to break the symmetry on the *values* actually assigned rather than force an arbitrary slot order: require the sequence of reviewers assigned to (Slot1, Slot2, Slot3) to be lexicographically ≤ any permutation of itself, e.g. by fixing Slot1's assigned reviewer to be the alphabetically-earliest reviewer appearing anywhere in the assignment (Slot1 ≤ Slot2 and Slot1 ≤ Slot3 under an alphabetical order $X < Y < Z$). This keeps exactly one representative per relabeling class – which reviewer sits where no longer matters – while still allowing every essentially different assignment of reviewers to slots.

Check Yourself 5.4

A model has four fully interchangeable Boolean "shift" variables S_1, S_2, S_3, S_4, with no constraint distinguishing them from each other. Write one lexicographic-ordering-style constraint that removes the pure relabeling symmetry among them.

Transfer — Where Else? 5 · Follow-through

Social golfer and BIBD (this unit's code) are not special cases – column/row symmetry of a 0/1 design matrix is one

of the most common patterns in constraint modeling, from BIBD's incidence matrix to timetabling's day-relabeling symmetry to routing's depot-relabeling symmetry in later courses; the lexicographic-ordering recipe you apply here is the same recipe you will reach for whenever a later problem's variables come in an interchangeable group. Next week keeps this whole five-week modeling toolbox – viewpoints, auxiliary variables, implied constraints, and now symmetry-breaking constraints – and asks whether the exact same model can be handed, unchanged, to a solver other than CP-SAT.

Chapter Summary

Summary

This chapter closed the course's five-week CSP core by asking a question none of Weeks 1–4 had asked outright: does a problem's own shape decide how hard it is, before any search code runs at all? Unit 1 answered "yes, sometimes completely" for tree-structured constraint graphs: directional arc consistency (DAC), swept once from the last variable back to the second in a tree-consistent ordering, guarantees by Freuder's tree theorem that a forward, root-to-leaves assignment never needs to backtrack – verified by hand on a 5-region tree map and cross-checked in code against a plain CP-SAT solve of the same instance. Unit 2 generalized that guarantee to graphs that are only *close* to a tree: cutset conditioning fixes a small cutset's values and solves the remaining tree backtrack-free for each fixed combination, while tree decomposition regroups the whole graph into overlapping bags connected in a tree and solves bag by bag; both reduce to, or generalize, Unit 1's same algorithm, and both are governed by one number, the graph's *treewidth* – the minimum induced width over all elimination orderings, computed by hand and in code as 1 for Unit 1's tree and 3 for a worked 5-node cutset example. Unit 3 placed both results inside the wider picture: CSP in general is NP-hard, since SAT (Week 1's own CSP-versus-SAT contrast) sits inside it as a special case, but bounded treewidth is a genuine tractable island – polynomial in the number of variables for a *fixed* treewidth, though exponential in that treewidth itself – and this worst-

case picture is exactly what Week 4's restarts, nogood learning and local search leave unchanged, since those techniques improve typical-case performance rather than lowering the worst case. Unit 4 turned from graph shape to solution shape: symmetry – a relabeling of variables, values, or both, that always turns one solution into another – multiplies a naive search's effort without adding anything genuinely new to find, and a lexicographic-ordering constraint fixes one canonical representative per symmetry class, verified on two real combinatorial-design instances: social golfer's branch count dropped from 2352 to 1454 and its conflict count from 147 to 25, and BIBD's branch count dropped from 1185 to 599, once the ordering constraint was added, real numbers captured from real CP-SAT runs, never invented. Next week keeps every modeling habit this five-week core has built – viewpoints, auxiliary variables, implied constraints, and now symmetry-breaking constraints – but asks whether the exact same model can be handed, unchanged, to a solver other than CP-SAT.

Check Yourself Answers

Question (Unit 1 – a path graph and backtrack-free assignment). A path is a special case of a tree: every internal node has exactly one parent and one child, and the two endpoints each have exactly one neighbour, so the graph P–Q–R–S is connected and contains no cycle. A tree-consistent ordering is simply the path's own order, P, Q, R, S: after the first variable P (the root), each later variable has exactly one earlier neighbour in the constraint graph – its parent – which is precisely the definition a tree-consistent ordering requires. Because this graph is tree-structured and this ordering is tree-consistent, Freuder's tree theorem applies without any further checking: once directional arc consistency has been achieved with respect to this same ordering, a forward assignment from P through S, choosing any remaining domain value at each step, is guaranteed backtrack-free – and this guarantee holds regardless of how many variables the path contains, since it follows from the graph's shape alone, not from its size.

Question (Unit 2 – a 6-cycle's cutset and induced width). Any single node of the 6-node cycle A–B–C–D–E–F–A is a valid cutset of size 1: removing any one node breaks the only cycle in the graph, leaving a path over the remaining five nodes, which is a tree. Taking A as that cutset variable and eliminating it first,

A's two remaining neighbours in the cycle – B and F – become connected by one fill-in edge, since eliminating a variable always closes its still-active neighbours into a clique; A's degree at the moment of its own elimination is therefore 2. Eliminating the rest of the graph along the resulting path B–C–D–E–F in order never produces a still-active neighbour count above 2 at any later step, since a path elimination never needs a fill-in edge once its first vertex is removed. The induced width for this ordering is therefore 2, one more concrete instance of a cutset of size 1 translating directly into a small, bounded induced width.

Question (Unit 3 – treewidth that grows with the instance). No, the tree-decomposition algorithm from Unit 2 does *not* guarantee a polynomial-time solve for this instance. Its guarantee is polynomial in the number of variables only for a treewidth that stays *fixed* as the instance grows – the algorithm's cost term is exponential in the treewidth, and that term only stays a harmless constant multiplier when the treewidth itself does not grow. Here the treewidth is stated to equal the number of variables minus one, i.e. it grows in lockstep with the instance size (essentially a complete graph, the densest possible constraint graph), so the "exponential in treewidth" term grows right along with the problem, giving no better guarantee than plain worst-case CSP solving already gives – Unit 2's tractability result is a statement about *fixed,* bounded treewidth, not about treewidth in general.

Question (Unit 4 – lexicographic ordering on four interchangeable shift variables). With no constraint distinguishing S_1, S_2, S_3, S_4 from one another, every one of the $4! = 24$ ways of relabeling which shift variable holds which of two interchangeable roles describes the exact same underlying real assignment, so an unmodified search would rediscover all 24 relabelings of every genuinely different solution separately. Requiring $S_1 \leq S_2 \leq S_3 \leq S_4$ fixes exactly one canonical ordering among those 24 relabelings: for any assignment of Boolean values to the four variables, at most one of its 24 relabelings can already satisfy a non-decreasing order, so the constraint keeps exactly one representative per symmetry class. Nothing about *which* values the four shifts genuinely take is removed by this constraint – only the redundant relabelings of an already-found assignment are excluded, exactly the same guarantee Unit 4's lexicographic-ordering constraints gave social golfer's and BIBD's own interchangeable groups.